

Project 2 – Real-Time Fraud Detection System

Sawyer Rideout

Introduction

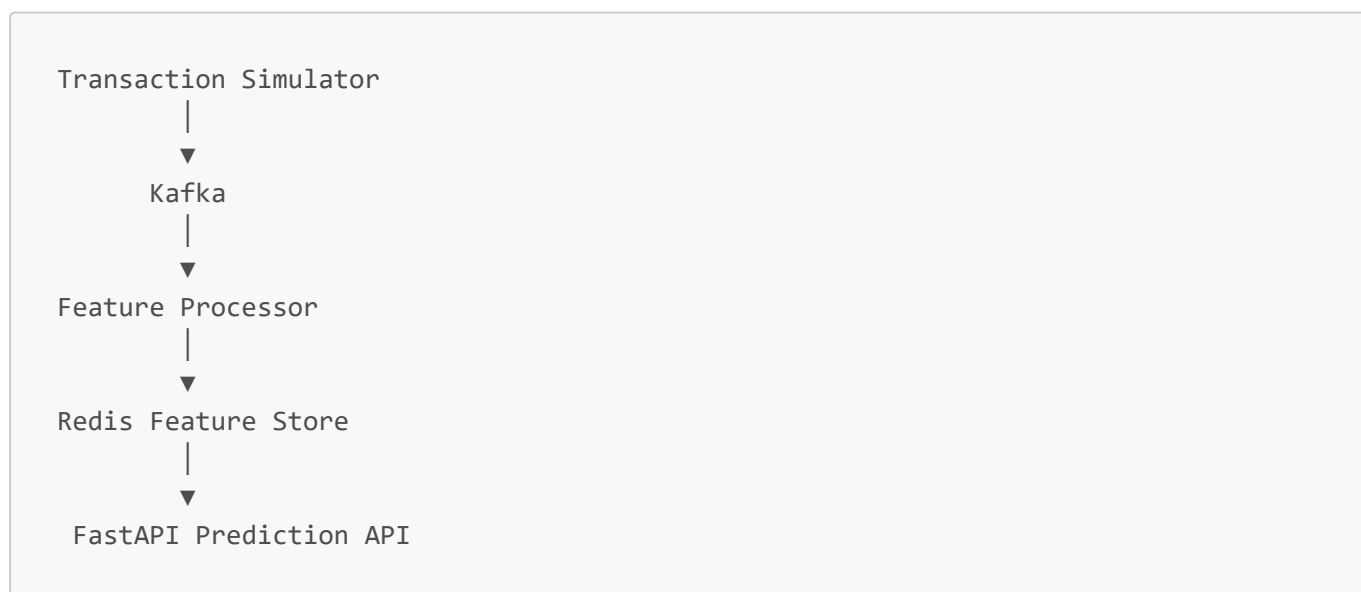
This project implements a real-time fraud detection system capable of processing streaming credit card transactions, computing customer features in real time, and serving fraud predictions through a containerized API. The overall system combines Apache Kafka for event streaming, Redis as a low-latency feature store, FastAPI for model serving, Docker for containerization, and nginx for blue-green deployment.

The primary objective of the project was to build an end-to-end streaming machine learning pipeline that continuously processes transactions while maintaining low prediction latency. Instead of calculating customer features during every prediction request, the system computes them continuously as transactions arrive. This allows the prediction API to retrieve precomputed customer information from Redis, reducing inference latency and separating streaming computation from model serving.

The completed system consists of four primary components:

- A **transaction simulator** continuously generates both historical and live transaction events.
- A **feature processor** consumes those events from Kafka, computes rolling customer statistics, and stores them in Redis.
- A **FastAPI prediction service** retrieves customer features from Redis and combines them with incoming transaction information to generate fraud predictions.
- **Docker Compose** orchestrates the application services, while an nginx-based blue-green deployment provides zero-downtime application updates.

The overall architecture is shown below.



This architecture separates event ingestion, feature computation, storage, and inference into independent services while maintaining real-time operation.

Part A – Streaming Pipeline & Feature Store

Streaming Architecture

The streaming pipeline follows a producer-consumer architecture centered around Apache Kafka. The transaction simulator acts as the producer by generating both a twenty-four-hour historical backfill and a continuous stream of live transaction events. These events are published to the `transactions` Kafka topic, allowing downstream consumers to process transactions independently.

The feature processor serves as the Kafka consumer. Rather than performing fraud prediction directly, it continuously reads transactions from Kafka, updates each customer's transaction history, computes rolling aggregate statistics, and writes the resulting features into Redis.

Separating feature computation from prediction provides two important advantages. First, the API no longer needs to calculate customer statistics during every prediction request, reducing response latency. Second, the streaming pipeline and prediction service become independent components, making the system easier to scale and maintain.

During implementation, the Kafka `transactions` topic was configured with three partitions and a replication factor of one. This satisfied the project requirement for a partitioned event stream while allowing the simulator and feature processor to communicate through Kafka in real time.

Evidence shown in docs -- `kafka-transactions.png`.

Rolling Window Feature Computation

The feature processor maintains an in-memory history of transactions for each customer. Whenever a new transaction is received, the transaction timestamp and amount are appended to that customer's event history.

Customer features are calculated using a rolling time window. For a given prediction time (`at_time`), only transactions satisfying the following condition are included:

```
(at_time - window_seconds) < event_time ≤ at_time
```

Using this rolling window, the processor computes two aggregate features for every customer:

- **transaction_count** – the total number of transactions within the active window.
- **avg_amount** – the average transaction amount within the same window.

If a customer has no transactions inside the window, the processor returns default values of:

- `transaction_count = 0`
- `avg_amount = 0.0`

Returning default values prevents prediction failures while ensuring every request receives a complete feature vector.

By continuously updating customer histories as new transactions arrive, feature computation occurs during event processing rather than during prediction requests. This design shifts computational work into the streaming pipeline, reducing API latency and allowing the prediction service to focus solely on inference.

Redis Feature Store

Redis serves as the low-latency feature store for the fraud detection system. Rather than recalculating customer statistics for every prediction request, the feature processor continuously computes rolling features and stores the latest values in Redis. This allows the prediction API to retrieve customer information using a single Redis lookup before performing inference.

Each customer's features are stored using the key format:

```
features:{customer_id}
```

The value associated with each key is a JSON document containing the customer's current rolling statistics. An example record is shown below for customer id CUST0057.

```
{
  "transaction_count": 118,
  "avg_amount": 136.23
}
```

Using Redis provides several advantages for this application. Customer features can be retrieved in only a few milliseconds, allowing prediction requests to remain fast even as transaction volume increases. Because Redis stores data in memory, feature retrieval is significantly faster than recalculating aggregates from the transaction stream during every prediction request.

To prevent stale customer information from remaining in memory indefinitely, every feature record is written with a configurable Time-To-Live (TTL). Once the TTL expires, Redis automatically removes the record. This ensures that inactive customers do not consume unnecessary memory while allowing active customers to continuously refresh their feature values as new transactions arrive.

The feature store also supports batch retrieval through Redis `MGET`. Instead of issuing one network request for every customer, multiple customer feature records can be retrieved using a single Redis round trip. This functionality is used by the batch prediction endpoint and reduces network overhead when multiple transactions are scored simultaneously.

During testing, the transaction simulator successfully generated customer activity that was consumed by the feature processor and written into Redis. Feature records were created for all simulated customers, confirming that the streaming pipeline and feature store were operating correctly.

End-to-End Streaming Verification

The completed streaming pipeline was verified by confirming successful communication between each component of the system.

First, the Kafka `transactions` topic was inspected to verify that transactions were being published correctly. The topic was configured with three partitions, satisfying the project requirements while allowing the producer and consumer to communicate through Kafka.

Next, Redis was queried directly using the Redis command-line interface. More than two hundred customer feature records were successfully stored using the expected `features:{customer_id}` key format. Individual records contained the expected rolling aggregates, including transaction count and average transaction amount.

Finally, the prediction API successfully retrieved customer features from Redis and combined them with incoming transaction information to generate fraud predictions. This verified the complete data flow through the system:



Successful end-to-end testing demonstrated that streaming ingestion, rolling feature computation, Redis storage, and API inference were functioning together as an integrated real-time fraud detection pipeline.

Part B – Model Serving & Containerization

FastAPI Prediction Service

The prediction service was implemented using FastAPI to provide a lightweight REST API for fraud detection. The application loads the fraud detection model and Redis feature store connection once during startup, preventing unnecessary initialization during every request. Keeping these resources in memory reduces request latency and improves overall throughput.

The API exposes four endpoints:

Endpoint	Purpose
GET /health	Returns the application health status.

Endpoint	Purpose
GET /model/info	Returns the currently loaded model version.
POST /predict	Scores a single transaction.
POST /predict_batch	Scores multiple transactions using a single batch feature lookup.

Before a prediction is generated, the API retrieves the customer's most recent features from Redis. If Redis is unavailable or no customer features exist, the application falls back to default values rather than failing the request. This allows the API to remain available even when feature data cannot be retrieved.

The transaction information and customer features are combined into a single feature dictionary before being passed to the fraud detector. The detector returns a fraud probability, binary prediction, and model version. The API also measures the total request latency using a high-resolution timer and returns that value as part of the response.

A typical prediction response is shown below.

```
{
  "transaction_id": "TXN-DEMO-001",
  "fraud_probability": 0.8,
  "is_fraud": 1,
  "model_version": "rule-based-fallback",
  "latency_ms": 20.468
}
```

The batch prediction endpoint follows the same process but retrieves customer features for all requested transactions using a single Redis batch operation. This reduces the number of network requests and improves efficiency when scoring multiple transactions simultaneously.

Containerization

The prediction service was containerized using Docker to provide a consistent runtime environment independent of the host operating system.

A multi-stage Docker build was used to reduce the final image size. The first stage installs all required Python dependencies, while the second stage copies only the installed packages and application files into a lightweight runtime image.

Several security and deployment best practices were incorporated into the Dockerfile:

- Multi-stage image build
- Python 3.11 slim base image
- Non-root application user
- HTTP health check
- Minimal runtime dependencies
- Exposed API port 8000

Running the application as a dedicated non-root user improves container security by limiting the privileges available to the application if it were ever compromised.

The Docker health check continuously verifies the `/health` endpoint. This allows Docker to determine whether the application is responding correctly without requiring external monitoring.

The application is started using Uvicorn.

```
uvicorn src.api.main:app --host 0.0.0.0 --port 8000
```

After building the image, the API successfully responded to health requests and prediction requests while running entirely inside the container.

Docker Compose Deployment

Docker Compose was used to orchestrate the complete fraud detection system. Instead of manually starting each application, Compose automatically creates a shared network and starts all required containers.

The deployment consists of five services:

Service	Responsibility
Kafka	Streaming transaction broker
Redis	Customer feature store
API	Fraud prediction service
Feature Processor	Kafka consumer and feature generator
Simulator	Kafka producer for historical and live transactions

Environment variables provide the connection information required for communication between services. For example, containers communicate with Kafka using the hostname `kafka:9092` and with Redis using `redis:6379`.

Using Docker Compose allows the complete application to be started with a single command:

```
docker compose up --build
```

This approach simplifies deployment while ensuring every service starts using a consistent configuration.

Blue-Green Deployment

To demonstrate zero-downtime deployment, a blue-green deployment environment was implemented using Docker Compose and nginx.

Two independent API containers were created:

- **Blue** deployment
- **Green** deployment

Only one version receives production traffic at any given time. nginx provides a stable endpoint on port **8080** while internally forwarding requests to the currently active API container.

Before traffic is switched, the deployment script verifies that the inactive API is healthy by checking its `/health` endpoint. If the health check succeeds, nginx reloads its configuration and begins routing new requests to the updated container.

Because nginx reloads its configuration without terminating existing worker processes, requests already being processed continue to completion while new requests are routed to the updated API. This allows deployments to occur without interrupting service availability.

Rollback is equally simple because both application versions remain running throughout the deployment. Executing the switch script a second time immediately redirects traffic back to the previous version without rebuilding or restarting containers.

During testing, repeated requests were continuously sent to the nginx endpoint while traffic was switched between the blue and green deployments. Every request returned HTTP status code **200**, confirming that the deployment completed without dropping requests.

Performance Evaluation

Performance Testing

System performance was evaluated using the provided benchmarking script:

```
python tests/test_performance.py --n 1000 --url http://localhost:8000
```

The benchmark submitted one thousand prediction requests to the API while recording throughput and latency statistics. Two configurations were evaluated during development. The initial configuration used a single Uvicorn worker, while the final configuration used two workers.

The final configuration produced the results shown below.

Metric	Result
Requests	1000
Errors	0
Throughput	166.6 requests/sec
p50 Latency	5.0 ms
p95 Latency	12.45 ms

Metric	Result
p99 Latency	17.29 ms
Maximum Latency	25.43 ms

All one thousand requests completed successfully with no prediction failures or HTTP errors. The p95 latency remained well below 100 milliseconds, indicating that the API can respond quickly even under sustained request volume.

Bottleneck Analysis

The primary bottleneck observed during testing was request concurrency within the API server rather than feature retrieval or prediction. Customer features were already being precomputed by the feature processor and stored in Redis, so prediction requests only needed to retrieve the latest feature values before performing inference.

The fraud detector itself also contributed very little computational overhead because inference was performed using the lightweight baseline model provided with the project. As a result, most request latency was associated with HTTP request handling rather than model execution.

Redis performed well throughout testing and consistently returned customer feature records with very low latency. Kafka was likewise not a limiting factor because feature computation occurred asynchronously before prediction requests reached the API.

Optimization

One optimization was evaluated by increasing the number of Uvicorn worker processes serving the FastAPI application.

Initially, the API was configured to run with a single worker. After increasing the configuration to two workers, the benchmark was repeated using the same workload. Throughput increased while latency decreased across every reported percentile.

Compared with the single-worker configuration, the two-worker deployment achieved approximately a twenty-five percent increase in throughput while reducing p95 latency by roughly twenty-three percent and p99 latency by approximately one-third. These results indicate that the application was limited primarily by request concurrency rather than computational complexity.

Because customer features were already stored in Redis and prediction itself required relatively little computation, increasing the number of workers allowed multiple requests to be processed simultaneously with minimal additional overhead.

Conclusion

This project successfully implemented a complete real-time fraud detection pipeline using Kafka, Redis, FastAPI, Docker, and nginx. Streaming transaction events were continuously processed by Kafka, customer

features were computed in real time by the feature processor, and Redis provided low-latency storage for the resulting feature vectors. The prediction API retrieved these features and combined them with incoming transaction data to generate fraud predictions while maintaining low response latency.

The application was containerized using a multi-stage Docker build and deployed using Docker Compose. A blue-green deployment strategy was implemented using nginx, allowing traffic to be switched between two independent API instances without interrupting service availability. During testing, repeated requests continued to return successful responses while traffic was switched, demonstrating zero-downtime deployment and rapid rollback capability.

Performance testing showed that the system could process approximately 194 prediction requests per second with a p95 latency of approximately 10 milliseconds while maintaining a zero percent error rate. Increasing the number of API worker processes further improved throughput and reduced latency, demonstrating that the architecture can be scaled without significant modification.

Overall, the completed system satisfies the project objectives by combining streaming data processing, low-latency feature storage, containerized model serving, and zero-downtime deployment into a single end-to-end fraud detection platform.